VideoJam — Technical Specification

Version: 1.0
Date: 2026-02-19
Status: Draft
Prepared by: Technical Architect (AI-assisted)

Table of Contents

1. System Overview
2. Architecture
3. Component Breakdown
4. Data Models
5. Show File Format
6. Audio Subsystem
7. Video Subsystem
8. Synchronisation Strategy
9. Display Management
10. Two-Button Control Model
11. UI Design (Operator Interface)
12. Show & Setlist Management
13. State Machine
14. Error Handling & Resilience
15. Security Considerations
16. Performance Targets
17. Dependencies & Third-Party Libraries
18. Build & Distribution
19. Testing Strategy

1. System Overview

VideoJam is a Windows 11 desktop application for live musical performance. It loads a pre-built show (setlist), allows the operator to configure audio levels before performance, and then delivers reliable fire-and-forget playback of synchronised audio stems and video files across multiple hardware displays.

Primary design constraints driving every architectural decision:

- **Reliability over features** — a failure in front of a live audience is unacceptable
- **Audio must never stop due to a video failure** — the audio and video subsystems are architecturally isolated
- **≤10ms start-time synchronisation** across all audio stems and all video streams
- **Fire-and-forget** — once GO is pressed, the operator does nothing until the song ends

2. Architecture

2.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│  VideoJam Process (.NET 10 / WPF)                                 │
│                                                                   │
│  ┌──────────────────────┐   ┌──────────────────────────────┐    │
│  │   Operator UI Layer  │   │       Playback Engine        │    │
│  │   (WPF — Display 0)  │   │                              │    │
│  │                      │   │  ┌────────────────────────┐  │    │
│  │  • Setlist panel     │   │  │  Audio Subsystem       │  │    │
│  │  • Level mixer       │◄──┤  │  (NAudio + WASAPI)     │  │    │
│  │  • Transport state   │   │  │                        │  │    │
│  │  • Show management   │   │  │  MixingSampleProvider  │  │    │
│  └──────────────────────┘   │  │  ├─ AudioFileReader ×N │  │    │
│                             │  │  │  (stems + video audio)│  │    │
│                             │  │  └─ WasapiOut (single)  │  │    │
│                             │  └────────────────────────┘  │    │
│                             │                              │    │
```

```
│                          │  ┌──────────────────────────┐  │  │
│                          │  │  Video Subsystem         │  │  │
│                          │  │  (LibVLCSharp)           │  │  │
│                          │  │                          │  │  │
│                          │  │  VlcDisplayWindow ×N     │  │  │
│                          │  │  (one per active display)│  │  │
│                          │  └──────────────────────────┘  │  │
│                          │                                │  │
│                          │  ┌──────────────────────────┐  │  │
│                          │  │  Sync Coordinator        │  │  │
│                          │  │  (start-time orchestration)│  │  │
│                          │  └──────────────────────────┘  │  │
│                          └────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
        │                                │
    ┌───▼───┐                    ┌──────▼──────┐
    │ WASAPI │                    │ Display Windows │
    │ Stereo │                    │ (HWND → libvlc) │
    │ Output │                    │ Display 1 … N   │
    └───────┘                    └─────────────────┘
```

2.2 Key Architectural Decisions

Decision	Choice	Rationale
Audio/video coupling	**Fully decoupled**	A/V failures are isolated; audio never stops for any video reason
Audio sync method	**Single WasapiOut with MixingSampleProvider**	All stems rendered in one callback — sample-accurate inter-stem sync by design
Video audio routing	**All audio through NAudio**	Video files played with VLC audio disabled; audio track decoded by NAudio's MediaFoundationReader
Start-time sync	**Shared high-resolution timestamp**	NAudio start timestamped via `Stopwatch`; VLC instances signalled immediately after
Per-display rendering	**Separate borderless WPF Window per display**	Each window provides an HWND to a dedicated LibVLC instance

Decision	Choice	Rationale
Show file format	**JSON**	Human-readable, portable, editable with any text editor

---

3. Component Breakdown

3.1 Component Map

```
VideoJam/
├── App.xaml / App.xaml.cs          — Application entry point; display detection on startup
├── UI/
│   ├── MainWindow.xaml/.cs         — Operator interface (setlist, levels, transport)
│   └── VlcDisplayWindow.xaml/.cs   — Full-screen video window (one instance per display)
├── Engine/
│   ├── PlaybackEngine.cs           — Orchestrates audio + video start/stop/pause
│   ├── SyncCoordinator.cs          — Manages start-time synchronisation sequence
│   ├── AudioEngine.cs              — NAudio pipeline: loads stems, manages mixer, drives WasapiOut
│   ├── VideoEngine.cs              — Manages LibVLC instances and VlcDisplayWindows
│   └── DisplayManager.cs           — Enumerates hardware displays; maps to Screen objects
├── Model/
│   ├── Show.cs                     — Root model: ordered song list + global config
│   ├── Song.cs                     — Song folder path + per-channel levels + routing overrides
│   ├── AudioChannel.cs             — Channel identity, level (0.0–1.0), mute state
│   ├── DisplayRoute.cs             — Suffix → display index mapping
│   └── AppState.cs                 — Runtime state (current song, playback state, etc.)
├── Services/
│   ├── ShowFileService.cs          — Serialise/deserialise .show files (System.Text.Json)
```

```
│   ├── SongScanner.cs              — Scans a song folder; classifies files by type
and suffix
│   └── PathResolver.cs             — Resolves relative paths against the .show file
location
└── Input/
    └── HotkeyService.cs            — Global keyboard hook for Button A / Button B
```

3.2 Component Responsibilities

`PlaybackEngine`

The central controller for a performance. Owns the current playback state and coordinates `AudioEngine`, `VideoEngine`, and `SyncCoordinator`. Exposes three operations: `Cue(song)`, `Go()`, `StopAndRewind()`. All other components are passive; only `PlaybackEngine` drives them.

`SyncCoordinator`

Responsible for the start-time synchronisation sequence (see Section 8). Records the precise timestamp when audio begins and issues the video start signal. Not involved after streams are running.

`AudioEngine`

Owns the NAudio pipeline. On `Load(song)`:

1. Creates an `AudioFileReader` (or `MediaFoundationReader`) for each audio channel
2. Wraps each in a `VolumeSampleProvider` to apply the channel's configured level
3. Composes all providers into a single `MixingSampleProvider`
4. Attaches the mix to a `WasapiOut` instance (not yet started)

On `Play()`: calls `WasapiOut.Play()` and returns the `Stopwatch` timestamp of that call.
On `Pause()`: calls `WasapiOut.Pause()`.
On `Stop()`: calls `WasapiOut.Stop()` and disposes all readers, resetting to the beginning of the song.

`VideoEngine`

Owns all `LibVLC` and `MediaPlayer` instances and their associated `VlcDisplayWindow` objects. On `Load(song)`:

- Opens each assigned video file in its corresponding VLC MediaPlayer
- Positions each player to time 0, buffers but does not play

On `Play(startTimestamp)`: signals each `MediaPlayer.Play()` as close to simultaneously as possible.
On `Pause()` / `Stop()`: delegates to each MediaPlayer.
On display disconnect: catches the event, stops affected VLC instances, logs the error. Does not notify `AudioEngine`.

`DisplayManager`

Enumerates connected displays via `System.Windows.Forms.Screen.AllScreens` on startup. Assigns display indices (0 = primary/laptop, 1–3 = external in enumeration order). Creates and positions `VlcDisplayWindow` instances to cover each physical screen. Does not handle hot-plug events — display topology is fixed at launch.

`SongScanner`

Given a song folder path, enumerates files and classifies them:

- Audio stems: `.wav`, `.mp3`, `.aiff` → `AudioChannel` records
- Video files: `.mp4` with recognised suffix → `VideoFile` records with resolved display index
- Unrecognised files: silently ignored

Returns a `SongManifest` (runtime object, not persisted) describing all channels and video assignments for that song.

`ShowFileService`

Serialises/deserialises `Show` objects to/from JSON. Uses `System.Text.Json` with source generation for trim-safe, AOT-compatible serialisation. Validates that required fields are present on load; surfaces validation errors as typed exceptions.

`PathResolver`

All paths in `.show` files are stored relative to the `.show` file location. `PathResolver` takes a relative path and the `.show` file's directory and returns an absolute path. Used whenever accessing song folders or fallback images.

`HotkeyService`

Registers a low-level keyboard hook (`SetWindowsHookEx WH_KEYBOARD_LL`) to capture Button A and Button B key presses globally, regardless of which window has focus. Default bindings: `Space` → Button A (GO), `Escape` → Button B (STOP/REWIND). Bindings are configurable in app settings.

---

4. Data Models

4.1 Runtime Models (in-memory only)

```
// Resolved at runtime from a song folder scan — never persisted
record SongManifest(
    string SongName,
    string FolderPath,
    IReadOnlyList<AudioChannelManifest> AudioChannels,
    IReadOnlyList<VideoFileManifest> VideoFiles
);

record AudioChannelManifest(
    string FilePath,       // absolute path
    string ChannelId,      // relative filename, used as key in Show file
    AudioChannelType Type  // Stem | VideoAudio
);

record VideoFileManifest(
    string FilePath,       // absolute path
    int DisplayIndex,      // resolved display index
    string Suffix          // e.g. "_lyrics"
);

enum AudioChannelType { Stem, VideoAudio }
```

4.2 Persisted Models (serialised to `.show` file)

```
class Show
{
    int Version { get; set; }                          // schema version, currently
1
    List<SongEntry> Songs { get; set; }
    Dictionary<string, int> GlobalDisplayRouting { get; set; }  // suffix → display
index
    Dictionary<int, string> FallbackImages { get; set; }        // display index →
relative PNG path
}

class SongEntry
{
    string FolderPath { get; set; }                    // relative to .show file
    string Name { get; set; }                          // display name (defaults to
folder name)
    Dictionary<string, int> DisplayRoutingOverrides { get; set; }  // per-song
overrides
    Dictionary<string, ChannelSettings> Channels { get; set; }     // channelId →
settings
}

class ChannelSettings
{
    float Level { get; set; }    // 0.0 to 1.0, default 1.0
    bool Muted { get; set; }     // default false (true for VideoAudio channels)
}
```

4.3 App State (runtime, not persisted)

```
enum PlaybackState { Idle, Cued, Playing, Paused }

class AppState
{
    Show LoadedShow { get; }
    string ShowFilePath { get; }
    int CuedSongIndex { get; }
    PlaybackState PlaybackState { get; }
    SongManifest? CuedSongManifest { get; }   // null if not yet scanned
}
```

---

5. Show File Format

Show files use the `.show` extension and contain UTF-8 encoded JSON. All file paths are relative to the directory containing the `.show` file.

5.1 Example `.show` file

```
{
  "version": 1,
  "globalDisplayRouting": {
    "_lyrics": 0,
    "_visuals": 1,
    "_stage": 2,
    "_rear": 3
  },
  "fallbackImages": {
    "0": "../assets/fallback_laptop.png",
    "1": "../assets/fallback_audience.png"
  },
  "songs": [
    {
      "folderPath": "../songs/01_OpeningTrack",
      "name": "Opening Track",
```

```
        "displayRoutingOverrides": {},
        "channels": {
          "drums.wav":                  { "level": 0.9, "muted": false },
          "bass.wav":                   { "level": 1.0, "muted": false },
          "keys.wav":                   { "level": 0.7, "muted": false },
          "crowd_fx.wav":               { "level": 0.5, "muted": false },
          "opening_visuals.mp4:audio":  { "level": 0.0, "muted": true  }
        }
      },
      {
        "folderPath": "../songs/02_AllInVideo",
        "name": "All-In-Video Song",
        "displayRoutingOverrides": {
          "_visuals": 1
        },
        "channels": {
          "performance.mp4:audio": { "level": 1.0, "muted": false }
        }
      }
    ]
  }
```

5.2 Channel ID Convention

Source	Channel ID format	Example
Audio stem file	Filename only (no path)	`drums.wav`
Audio track in video file	`{filename}:audio`	`opening_visuals.mp4:audio`

5.3 Display Routing Resolution

For each video file in a song folder:

1. Check per-song `displayRoutingOverrides` for a matching suffix — use if found
2. Fall back to `globalDisplayRouting` for a matching suffix — use if found
3. If no match — video file is ignored (not routed to any display)

Display index 0 is always the primary display (laptop screen). Indices 1–3 are external displays in the order enumerated by Windows.

5.4 Path Portability

The `.show` file and the song folders it references must share a common root (e.g. on the same USB drive or in the same directory tree). The operator is responsible for maintaining the relative folder layout when copying between machines.

---

6. Audio Subsystem

6.1 Pipeline

```
AudioFileReader (stem 1) ──► VolumeSampleProvider ──┐
AudioFileReader (stem 2) ──► VolumeSampleProvider ──┤
MediaFoundationReader    ──► VolumeSampleProvider ──┤──► MixingSampleProvider ──►
WasapiOut
  (video audio 1)                                   │         (single instance)
  │
MediaFoundationReader    ──► VolumeSampleProvider ──┘
  │
  (video audio 2)
   ▼

Stereo Output
                                                                          (system
audio device)
```

6.2 Format Handling

Format	Reader Class	Notes
WAV	`AudioFileReader` (NAudio)	Native NAudio support
MP3	`AudioFileReader` (NAudio)	Via built-in ACM decoder
AIFF	`AiffFileReader` (NAudio)	Native NAudio support
MP4 audio track	`MediaFoundationReader` (NAudio)	Decodes via Windows Media Foundation

All sources are resampled to a common format (44100 Hz, 16-bit or 32-bit float, stereo) by NAudio's sample conversion chain before entering the `MixingSampleProvider`.

6.3 WASAPI Configuration

- **Mode:** Shared (not exclusive) — avoids locking the audio device; tolerant of other Windows audio activity
- **Latency target:** 50ms buffer (balances latency against stability; adjustable in app settings if needed)
- **Device:** Default Windows audio output device (no device selection UI in MVP)

6.4 Level Control

Each channel's `VolumeSampleProvider` has its `Volume` property set from the `ChannelSettings.Level` float (0.0–1.0) at load time. Levels are not adjustable during playback (the mixer UI controls are disabled in the `Playing` and `Paused` states).

6.5 Pre-loading

All audio files are opened and their pipelines constructed during `Cue(song)`, which occurs when the operator selects a song — before they press GO. This ensures no file I/O or pipeline construction occurs on the hot path at the moment of playback start.

---

7. Video Subsystem

7.1 Per-Display Architecture

Each physical display beyond the primary (operator) screen has:

- One **`VlcDisplayWindow`** — a borderless, topmost WPF `Window` positioned to exactly cover the physical monitor
- One **`LibVLC`** instance
- One **`MediaPlayer`** instance associated with that `LibVLC` instance
- The window's HWND is passed to the `MediaPlayer` as its render surface

The primary display (Display 0 / laptop screen) behaves differently: between songs it shows the operator UI (`MainWindow`). When a song with a `_lyrics` video begins, `MainWindow` hides and a `VlcDisplayWindow` for Display 0 is shown full-screen. When the song ends, the `VlcDisplayWindow` is hidden and `MainWindow` is restored.

7.2 Fallback PNG Display

When no video is assigned to a display for the current song (or the app is in the `Idle` or `Cued` state), the corresponding `VlcDisplayWindow` displays the show's configured fallback PNG for that display. This is implemented by rendering the image directly in the WPF window (an `<Image>` element behind the VLC render surface) rather than via VLC.

7.3 VLC Configuration

Each `MediaPlayer` is configured with:

- `--no-audio` — audio output completely disabled; all audio routed through NAudio
- `--no-osd` — no on-screen display overlays
- `--loop` disabled — play once and stop
- Hardware-accelerated H.264 decode enabled (VLC default on Windows)

7.4 End-of-Song Detection

End-of-song is detected via NAudio (the authoritative timeline), not VLC. When the `MixingSampleProvider` has no more samples to provide, `WasapiOut` raises its `PlaybackStopped` event. `PlaybackEngine` handles this event and initiates the end-of-song transition. Video players are stopped as part of this transition.

7.5 Display Disconnect Handling

`DisplayManager` subscribes to the `SystemEvents.DisplaySettingsChanged` event. If a display disappears mid-performance:

1. The affected `VlcDisplayWindow` is closed and its `MediaPlayer` disposed
2. The loss is logged
3. `AudioEngine` is not notified — audio continues uninterrupted
4. No user-facing error is shown (would be a distraction during performance)

8. Synchronisation Strategy

8.1 Inter-Stem Audio Sync

All audio stems are channels in a single `MixingSampleProvider` driven by a single `WasapiOut`. They share one audio clock. Synchronisation is exact by construction — there is nothing to coordinate.

8.2 Audio-to-Video Start Sync

The synchronisation sequence on GO:

```
1. [Pre-GO, during Cue]
   - All audio readers constructed and positioned at t=0
   - All VLC MediaPlayers opened, buffered, positioned at t=0
   - All VlcDisplayWindows visible and showing fallback PNG

2. [GO pressed]
   - WasapiOut.Play() called  ← audio starts rendering
   - t_start = Stopwatch.GetTimestamp() recorded immediately after

3. [Synchronous, same thread, ~microseconds later]
   - foreach MediaPlayer: MediaPlayer.Play() called
   - Δt between audio start and last VLC start signal ≈ N × ~10µs
     (N = number of video players; in practice Δt << 1ms)

4. [VLC internally]
   - VLC receives Play() signal and begins presenting decoded frames
   - Frame 0 presentation target: t_start + decode_buffer_time
   - Actual first frame on screen: within one frame period of t_start
     (≤16ms at 60fps; ≤41ms at 24fps — within the stated tolerance)
```

Why this is acceptable: The 10ms sync requirement (NFR-001) applies to stream start alignment. The requirements document (Assumption 7) explicitly accepts single-frame video presentation jitter as long as audio streams remain tightly aligned. Audio inter-stem sync is exact; audio-to-video sync is within one frame period.

8.3 Ongoing Drift

After streams are started, each subsystem maintains its own clock:

- Audio: driven by the WASAPI hardware clock (highly stable)
- Video: driven by VLC's internal presentation clock, derived from frame timestamps in the H.264 stream

Drift over a 3–4 minute song is expected to be imperceptible. If it becomes a concern in practice, a clock-slaving mechanism can be added in a future phase (VLC supports external clock input via `libvlc_media_player_set_time`).

---

9. Display Management

9.1 Display Enumeration

On startup, `DisplayManager` calls `System.Windows.Forms.Screen.AllScreens` to enumerate all connected displays. It assigns indices in enumeration order:

- Index 0 — Primary display (marked `IsPrimary == true` in Windows)
- Index 1, 2, 3 — Additional displays in enumeration order

The enumeration result is fixed for the lifetime of the application. If displays are connected or disconnected after launch, the app does not re-enumerate.

9.2 Window Positioning

Each `VlcDisplayWindow` is positioned using the `Bounds` property of its corresponding `Screen` object:

```
window.Left   = screen.Bounds.Left   / dpiScale;
window.Top    = screen.Bounds.Top    / dpiScale;
window.Width  = screen.Bounds.Width  / dpiScale;
window.Height = screen.Bounds.Height / dpiScale;
window.WindowStyle = WindowStyle.None;
window.ResizeMode  = ResizeMode.NoResize;
window.Topmost     = true;
```

DPI scaling is accounted for by dividing pixel coordinates by the per-monitor DPI scale factor.

9.3 Single-Display Mode

If only one display is connected, `DisplayManager` creates no `VlcDisplayWindow` instances for external displays. Video files routed to indices 1–3 are silently ignored. The primary display (`MainWindow` + optional Display 0 `VlcDisplayWindow`) operates normally.

---

10. Two-Button Control Model

10.1 Button Definitions

Button	Default Key	Action
Button A (GO)	`Space`	See state machine below
Button B (STOP/REWIND)	`Escape`	See state machine below

Both keys are captured via a global low-level keyboard hook, so they work regardless of which window has focus.

10.2 Button Behaviour by State

Current State	Button A pressed	Button B pressed
`Idle`	No action	No action
`Cued`	→ `Playing` (start playback)	→ `Idle` (de-cue)
`Playing`	No action	→ `Paused`
`Paused`	No action	→ `Cued` (rewind to beginning)

10.3 Song Advance

After a song ends naturally (`PlaybackStopped` event from NAudio), the app:

1. Transitions all displays to fallback PNG state
2. Advances `CuedSongIndex` to the next song (if not at the end of the setlist)
3. Enters `Cued` state

4. The operator can then press GO to start the next song, or click any song in the setlist UI to change the selection

If the setlist is at the last song and it completes, the app enters `Idle` state with no song cued.

---

11. UI Design (Operator Interface)

11.1 Layout

```
┌──────────────────────────────────────────────────────────┐
│ VideoJam                          [Show Name]  [Save]    │
├──────────────────────────┬──────────────────────────────┤
│   SETLIST                │   MIXER                      │
│                          │                              │
│  ► 01  Opening Track     │  drums.wav        [====|  ] │
│    02  Song Two          │  bass.wav         [======] │
│    03  Song Three        │  keys.wav         [===|   ] │
│    04  All-In-Video      │  crowd_fx.wav     [==|    ] │
│    05  Closer            │  visuals.mp4:audio [M      ]│
│                          │                              │
│  [+ Add Song]            │  (locked during playback)    │
│                          │                              │
├──────────────────────────┴──────────────────────────────┤
│  STATE: CUED — "Opening Track"          [GO: Space]      │
└──────────────────────────────────────────────────────────┘
```

11.2 Setlist Panel

- Displays all songs in show order, numbered
- Currently cued song highlighted (arrow indicator)
- Drag-and-drop reordering (WPF drag-drop)
- Click any song to cue it (only when not in `Playing` state)
- "Add Song" button opens a folder browser

11.3 Mixer Panel

- Displays all audio channels for the currently selected/cued song
- Each channel shows: filename, level slider (0–100%), mute toggle
- Video audio channels shown with `[M]` indicator (muted by default)
- All controls disabled during `Playing` and `Paused` states
- Changes are reflected immediately in the NAudio pipeline on next `Cue()`

11.4 Status Bar

- Displays current playback state in plain language
- Shows the name of the cued song
- Reminds the operator of the GO key binding
- Displays a non-intrusive warning if a display disconnected during performance

11.5 Show Management

- **New Show:** File → New (prompts to save if unsaved changes exist)
- **Open Show:** File → Open (`.show` file picker)
- **Save Show:** File → Save / Ctrl+S
- **Save As:** File → Save As

---

12. Show & Setlist Management

12.1 Adding a Song

1. Operator clicks "Add Song"
2. Folder browser opens; operator selects a song folder
3. `SongScanner` scans the folder and returns a `SongManifest`
4. A `SongEntry` is created with:
    - `FolderPath` : relative path from the `.show` file location to the song folder
    - `Name` : the folder's name
    - `Channels` : one `ChannelSettings` per detected audio channel, with defaults (`level: 1.0` ,

    `muted: false` for stems; `level: 0.0`, `muted: true` for video audio)
  - `DisplayRoutingOverrides`: empty
5. The song is appended to the setlist

12.2 Path Relativisation

When a song is added, `PathResolver` converts the absolute folder path to a path relative to the current `.show` file location. If no `.show` file has been saved yet, the path is stored as absolute and relativised when the show is first saved.

12.3 Unsaved Changes

A dirty flag (`bool _hasUnsavedChanges`) is set whenever any model change occurs. The title bar shows an asterisk when dirty. The app prompts to save on close or new/open if dirty.

---

13. State Machine

```
┌──────────────────────────────────┐
│              IDLE                │
│  No song cued. Displays show     │
│  fallback PNGs. Operator UI visible.│
└──────────┬───────────────────────┘
           │  Click song in setlist / auto-advance
           ▼
┌──────────────────────────────────┐
│              CUED                │
│  Song selected. Audio pipeline   │
│  loaded. VLC instances preloaded. │
│  Operator can adjust levels.     │
└──────────┬───────────────────────┘
           │  Button A (GO)
           ▼
┌──────────────────────────────────┐
│             PLAYING              │◄──────────┐
│  All streams running. Operator UI │           │
│  hidden (Display 0 showing video  │  Button B │
│  or fallback). Mixer locked.     │  pressed once │
```

```
└──────┬──────────────────────────────┘          │
       │  Song ends naturally (NAudio             │
       │  PlaybackStopped event)                  │
       ▼                                          │
┌──────────────────────────────────┐              │
│  CUED (next song)                │      ┌───────┴────────┐
│  or IDLE (end of setlist)        │      │    PAUSED      │
└──────────────────────────────────┘      │                │
                                          │ Button B again │
                                          │ → back to CUED │
                                          └────────────────┘
```

---

14. Error Handling & Resilience

14.1 Error Categories

Category	Examples	Response
File not found at load time	Song folder moved, stem file deleted	Show error in UI; prevent playback of affected song; do not crash
File format unreadable	Corrupt MP4, unsupported codec	Log error; skip file; show warning in mixer panel for affected channel
Audio device unavailable	WASAPI device lost	Show error; transition to `Idle`; do not attempt retry during performance
Display disconnect mid-performance	HDMI cable pulled	Stop affected VLC instance silently; audio continues; log event; show unobtrusive status bar note
VLC instance failure	Codec crash, OOM	Dispose affected VLC instance; log; audio continues
Show file parse error	Invalid JSON, missing fields	Show descriptive error dialog; do not load the show

14.2 Principles

- **Never crash silently.** All exceptions are caught at component boundaries and result in either a user-visible error (non-performance) or a log entry (mid-performance).
- **Audio is the last line of defence.** Code paths in `AudioEngine` avoid all I/O, file operations, and external calls during playback. The only thing that stops audio is an `AudioEngine` -internal failure or an explicit `Stop()` call.
- **Fail fast before the show.** Song validation (file existence, readability) is performed on `Cue(song)` , not on `Go()` . Errors are surfaced during setup, not during performance.

14.3 Logging

Application events and errors are written to a rolling log file at `%APPDATA%\VideoJam\logs\videojam.log` using `Microsoft.Extensions.Logging` with a file sink. Log level is `Information` by default. Log files are retained for 7 days.

---

15. Security Considerations

VideoJam has a minimal security surface:

- **No network connectivity** — the app is entirely offline; no inbound or outbound network calls
- **No user accounts or personal data** — nothing to protect
- **Local file access only** — reads media files and `.show` files from paths the operator explicitly selects; no directory traversal concerns
- **No elevated privileges required** — runs as a standard user; no UAC prompts
- **Dependency supply chain** — NuGet packages (NAudio, LibVLCSharp, VideoLAN.LibVLC.Windows) should be pinned to specific versions and reviewed on update

No further security controls are required.

---

16. Performance Targets

Target	Metric	How Achieved
Stream start sync	≤10ms across all audio and video streams	Single-call audio start; immediate sequential VLC signals
GO press response	Imperceptible (<50ms)	All media pre-loaded during Cue; no I/O on hot path
App startup to ready	<30 seconds	No heavy initialisation; VLC loaded lazily per song
Audio glitch-free	Zero dropouts during performance	50ms WASAPI buffer; no blocking calls on audio thread
Video decode	Hardware-accelerated H.264	VLC default on Windows with compatible GPU
Memory	No growth during performance	All media loaded at Cue time; no dynamic allocation during playback

---

17. Dependencies & Third-Party Libraries

Package	Version (pinned)	Purpose
`NAudio`	2.2.x	Audio file reading, mixing, WASAPI output
`LibVLCSharp`	3.x	.NET wrapper for libvlc
`VideoLAN.LibVLC.Windows`	3.x	Native libvlc binaries for Windows (x64)
`System.Text.Json`	(in-box .NET 10)	Show file serialisation
`Microsoft.Extensions.Logging`	8.x	Structured logging
`xunit`	2.x	Unit testing framework

Version pinning: All third-party packages are pinned to specific minor versions in `Directory.Build.props`. Patch updates are acceptable; minor/major updates require explicit review and testing.

---

18. Build & Distribution

18.1 Build Command

```
dotnet publish VideoJam.csproj \
  --configuration Release \
  --runtime win-x64 \
  --self-contained true \
  --output ./publish \
  -p:PublishSingleFile=false
```

`PublishSingleFile=false` is intentional — libvlc native binaries cannot be bundled into a single file and must remain as separate DLLs in the output directory.

18.2 Output Structure

```
VideoJam/              ← zip this folder
├── VideoJam.exe
├── VideoJam.dll
├── libvlc.dll
├── libvlccore.dll
├── plugins/           ← VLC codec plugins
│   └── ...
└── [.NET runtime DLLs]
```

The operator unzips this folder anywhere (USB drive, desktop, etc.) and double-clicks `VideoJam.exe`. No installer, no admin rights required.

18.3 Target Runtime

- `win-x64` only — no ARM64 build required
- Minimum OS: Windows 11 (any release)

---

19. Testing Strategy

19.1 Unit Tests (xUnit)

Unit tests cover all logic that does not require media playback:

Area	What is tested
`ShowFileService`	Serialisation round-trips; validation of malformed input; version field handling
`PathResolver`	Relative path construction; round-trip resolution; edge cases (same directory, deep nesting)
`SongScanner`	File classification by extension and suffix; handling of unknown files; empty folder
`DisplayRoute` resolution	Global routing; per-song overrides; suffix-not-found case
`ChannelSettings` defaults	New song defaults (level=1.0, muted=false for stems; muted=true for video audio)
State machine transitions	Valid and invalid transitions; end-of-setlist behaviour

Test project: `VideoJam.Tests` (sibling project in the solution).

19.2 Manual Integration Testing

All media playback testing is performed manually using a known-good test set maintained by the operator. The test set should include:

- A song with multiple audio stems only (no video)
- A song with audio stems + video files for multiple displays
- A song with video audio channel unmuted and no separate stems
- A song folder with unrecognised files (to verify they are silently ignored)
- A `.show` file opened from a different machine (to verify path resolution)

19.3 Performance Testing

Sync accuracy is verified manually using the operator's known-good test set: a precisely-aligned test video with an audio cue at frame 0. Observed drift at the end of a full-length song is noted.

19.4 What Is Not Tested Automatically

- Audio output quality
- Video rendering correctness
- Multi-display synchronisation
- Hardware-specific behaviour (GPU decode, WASAPI device enumeration)

These are tested manually and are inherently environment-dependent.

---

20. Revision History

Version	Date	Author	Changes
1.0	2026-02-19	Technical Architect (AI-assisted)	Initial draft